

Anatomy of an Agent

Last lecture (L1), we talked about **agents in the abstract**:
what they are, what perception and observation mean,
and how the concept evolved.

This lecture connects those ideas to code.



The obvious question

***"If Claude
Code/Codex/Cursor/Copilot/...
can write an agent for me in
60 seconds, why do I need to
learn any of this?"***

Universal coding agents are indeed great prototyping tools. And yes, you can achieve great results with them. And you should!

However, they are only as good as your understanding of specifications, architecture and testing technics.

The five agents you will meet today

#	AGENT	NEW THING IT ADDS
1	Reflex agent	The bare architecture. No LLM at all.
2	Minimal LLM agent	LLM replaces hand-written rules.
3	Multi-step LLM agent	Loop + one tool. Agent talks to the world.
4	Baby memory agent	State across percepts, on disk.
5	Memory toddler agent	More comprehensive tools.

The task

A simple agent that receives a web link and answers what the link is about.

1. The reflex agent

The absolute simplest thing that qualifies as an agent. Just perception, rules, action, loop.

Perception: `perceive()`

reflex_agent.py

```
1 def perceive(url: str) -> dict:
2     parsed = urlparse(url)
3     return {
4         "scheme": parsed.scheme,           # "https"
5         "domain": parsed.netloc.lower(),    # "youtube.com"
6         "path":   parsed.path.lower(),     # "/watch"
7     }
```

WHAT

Takes a raw URL string, returns a structured dict — the “percept.”

NOTICE

Perception is the **only** way information enters the agent. It's a bottleneck. Everything you don't extract here is invisible to the rest of the code.

Thinking: the rule table

reflex_agent.py

```
1 RULES = [  
2     (lambda p: p["scheme"] not in ("http", "https"),  
3         "not-a-web-link"),  
4     (lambda p: "youtube.com" in p["domain"] or "youtu.be" in p["domain"],  
5         "video"),  
6     (lambda p: "github.com" in p["domain"],  
7         "code"),  
8     (lambda p: "arxiv.org" in p["domain"] or p["path"].endswith(".pdf"),  
9         "paper"),  
10    (lambda p: "wikipedia.org" in p["domain"],  
11        "reference"),  
12    (lambda p: "x.com" in p["domain"] or "twitter.com" in p["domain"],  
13        "social"),  
14    (lambda p: "medium.com" in p["domain"] or "substack.com" in p["domain"],  
15        "blog"),  
16 ]
```

WHAT

A list of (condition, answer) pairs. This IS the agent's brain.

WHY

It's the direct implementation of R&N's *condition-action rules* from the classical simple-reflex-agent pseudocode.

WHAT TO NOTICE

Every rule was written by us, by hand.
New categories = new lines. No learning,
no generalization.

Thinking: think()

reflex_agent.py

```
1 def think(percept: dict) -> str:
2     for condition, action in RULES:
3         if condition(percept):
4             return action
5     return "unknown"
```

WHAT

Walk the rule list top-to-bottom. First match wins. Fall through to "unknown" if nothing matches.

WHY LIKE THIS

Nothing more is needed. The whole "brain" is a linear search over rules.

NOTICE

The smallest possible thinking block. No memory of past inputs. No sense of context.

Action: `act()`

reflex_agent.py

```
1 def act(decision: str, url: str) -> str:  
2     return f"[{decision}] {url}"
```

WHAT

Format a string. That's the "effect on the world."

WHY LIKE THIS

For the demo, the world is the terminal and the actuator is `print()`. Real agents replace this with tool calls, database writes, sending messages.

WHAT TO NOTICE

`act()` is separated from `think()` on purpose. Later we can swap `print` for `send_email` **without touching perception or thinking**. Separation of concerns is what makes the triad useful, not just descriptive.

The loop

reflex_agent.py

```
1 def agent_step(url: str) -> str:
2     percept = perceive(url)
3     decision = think(percept)
4     return act(decision, url)
5
6 while True:
7     url = input("> ")
8     if not url:
9         break
10    print(agent_step(url))
```

WHAT

`agent_step` = one tick of the loop. The outer `while True` = the ongoing life of the agent.

NOTICE

The loop is not `for url in list`. It's `while True: input()`.

Function vs agent

BATCH SCRIPT

```
1 for url in list_of_urls:
2     print(agent_step(url))
```

input → output → done. Dies.

AGENT

```
1 while True:
2     url = input("> ")
3     print(agent_step(url))
```

perceives → thinks → acts → continues to
exist

That “continues to exist” is what R&N’s definition captures. Our agent is idle between percepts — but it’s **alive**.

PEAS

PEAS	WHAT IT MEANS	IN OUR REFLEX AGENT	WHERE IN CODE
P — Performance measure	How we measure success	Fraction of URLs that receive a correct non-unknown label.	Defined externally
E — Environment	Where the agent lives	Terminal REPL. Stream of URL strings a user types, one line at a time.	Surrounds the Python process
A — Actuators	How the agent acts on E	<code>print()</code> — writes one line per URL	<code>print(agent_step(url))</code> inside <code>while True:</code>
S — Sensors	How the agent perceives E	<code>input()</code> — reads one line from stdin	<code>url = input("> ")</code> inside <code>while True:</code>

ReAct

REACT COMPONENT	WHAT THE PAPER MEANS	IN OUR REFLEX AGENT	PRESENT?
Thought	Internal reasoning about what to do next	Rule lookup in <code>think()</code> — “which condition matches this percept?”	✓ (trivial — a single lookup, no chain of thought)
Action	The concrete step chosen	Label string returned, then <code>print()</code> -ed to E	✓
Observation	Feedback from E after the action	✗ — the agent never looks at what happened after printing	Missing
Iteration loop	Repeat Thought → Action → Observation until done	✗ — one tick per percept, no re-examination inside a tick	Missing
Final Answer	Agent decides it's done and outputs the final result	Every <code>agent_step()</code> is immediately the final answer	✓ (trivial — first pass is the answer)

The environment

AXIS	OUR ENVIRONMENT	
Observability	Fully observable	Whole URL visible; nothing hidden.
Determinism	Deterministic	Same URL → same answer. Rules are pure.
Episodicity	Episodic	Each URL is independent; no history matters.
Dynamism	Static	Nothing changes while the agent thinks.
Discreteness	Discrete	Discrete inputs (strings), discrete outputs (labels).
Number of agents	Single-agent	No competitors.

CoALA — reflex agent

COMPONENT	PRESENT?	HOW
Working memory	×	Percept → decision in a single pass
Episodic LTM	×	No history
Semantic LTM	×	—
Procedural LTM	✓	The hand-coded RULES table
Internal actions	Retrieval only	Lookup in RULES
External actions	Dialog	Terminal in, terminal out

The minimal possible agent in CoALA terms: one procedural memory, no other modules, no loop.

2. Minimal LLM agent

Thinking: PROMPT_TEMPLATE

llm_agent.py

```
1 PROMPT_TEMPLATE = (  
2     "You are a URL classifier. Given the parts of a URL below, respond "  
3     "with EXACTLY ONE lowercase English word describing what kind of "  
4     "resource it is. Allowed words: video, code, paper, reference, "  
5     "social, blog, news, docs, shop, forum, unknown. "  
6     "Output the word only. No punctuation. No explanation.\n\n"  
7     "scheme: {scheme}\n"  
8     "domain: {domain}\n"  
9     "path: {path}"  
10 )
```

WHAT

A string template. The {scheme}, {domain}, {path} slots get filled with the percept fields.

WHY LIKE THIS

This prompt plays the SAME role that RULES played in the reflex agent.

WHAT TO NOTICE

We used to program the brain in Python. Now we program it in English.

Thinking: think()

llm_agent.py

```
1 def think(percept: dict) -> str:
2     prompt = PROMPT_TEMPLATE.format(**percept)
3     result = subprocess.run(
4         ["claude", "-p", prompt],
5         capture_output=True, text=True, timeout=120,
6     )
7     return result.stdout.strip().split()[0].strip(".!?",).lower()
```

WHAT

Fill the template. Shell out to `claude -p "<prompt>"`. Take the first word of the answer. Strip punctuation. Lowercase.

WHY LIKE THIS

`claude -p` = I route it to the headless Claude Code CLI. Here we can either route to external API or use a local model.

WHAT TO NOTICE

LLM output normalization: `.split()[0]`, `.strip(...)`, `.lower()` (the cost of a probabilistic brain:) you must clean its output before the rest of the code trusts it)

The interface is unchanged

reflex_agent.py = llm_agent.py

```
1 def think(percept: dict) -> str:  
2     ...
```

Input	<code>dict</code> — a percept
-------	-------------------------------

Output	<code>str</code> — a label
--------	----------------------------

Because the interface is identical, `perceive()`, `act()`, `agent_step()` don't need to know anything changed.

This is just a good engineering practice that can save you a lot of time and iterate fast.

PEAS

PEAS	IN MINIMAL LLM AGENT	SAME AS 1.2?
P — Performance	Fraction of URLs given a correct label. <code>unknown</code> still allowed but rarer (LLM has broader coverage than rules).	Almost same — same criterion, better hit rate expected
E — Environment	Terminal REPL. Stream of URL strings typed by a user.	✓ Identical
A — Actuators	<code>print()</code> — writes one line to the terminal	✓ Identical
S — Sensors	<code>input()</code> — reads one line from stdin	✓ Identical

ReAct

MODULE	IN MINIMAL LLM AGENT	SAME AS 1.2?
Perception	Parses URL string into <code>{scheme, domain, path}</code>	✓ Identical — <code>perceive(url)</code> copy-pasted
Reasoning / Decision	CHANGED. Builds a prompt from the percept, shells out to <code>claude -p</code> , normalizes the output token	✗ Different — see <code>think()</code> + <code>PROMPT_TEMPLATE</code>
Action formation	Builds string <code>[label] url</code>	✓ Identical — <code>act()</code> copy-pasted
Memory	Still none	✓ Still absent
Control loop	<code>while True: input() ... print()</code>	✓ Identical

The environment — minimal LLM agent

AXIS	OUR ENVIRONMENT	WHY
Observability	Fully observable	Whole URL visible; nothing hidden
Determinism	Deterministic (in the model)	The environment is the same terminal; the LLM changes the policy, not the world
Episodicity	Episodic	Each URL is independent
Dynamism	Static	Nothing changes while the agent thinks
Discreteness	Discrete	Strings in, labels out
Agents	Single-agent	Nobody else in the environment

Identical to the reflex agent: the LLM inside changes the policy, not the environment.

CoALA — minimal LLM agent

COMPONENT	PRESENT?	HOW
Working memory	Minimal	Percept as part of one LLM prompt
Episodic LTM	✗	No history
Semantic LTM	Implicit	In the LLM weights
Procedural LTM	✓	PROMPT_TEMPLATE instead of the RULES table
Internal actions	Reasoning	One LLM call
External actions	Dialog	Terminal

Main shift vs reflex: procedural memory moved from Python code into natural language.

Multi-step LLM agent with a tool

Now the agent can take several thinking steps per input, and it has one tool to reach into the world.

The tool: `fetch_url`

llm_agent.py

```
1 def fetch_url(url: str, max_chars: int = 4000) -> str:
2     try:
3         req = urllib.request.Request(
4             url,
5             headers={"User-Agent": "Mozilla/5.0 (compatible; LLMAgent/1.0)"},
6         )
7         with urllib.request.urlopen(req, timeout=15, context=_SSL_CTX) as response:
8             raw = response.read(max_chars * 4)
9             return raw.decode("utf-8", errors="replace")[:max_chars]
10    except Exception as e:
11        return f"[fetch failed: {e}]"
```

WHAT

Standard-library HTTP client (`urllib.request`). Downloads the first 4000 characters of the page. Returns the text.

WHAT TO NOTICE

Errors don't crash the agent — `fetch_url` returns `"[fetch failed: ...]"` as a string. The agent stays alive.

Tool registry

llm_agent.py

```
1 TOOLS = {  
2     "fetch_url": fetch_url,  
3 }
```

WHAT

A dict mapping tool name (string) to tool function.

WHY

The LLM will only ever say the **name** of a tool. The code looks the name up in this dict and calls the actual function.

WHAT TO NOTICE

Adding a tool = adding one line to this dict. Removing a tool = removing one line.

The system prompt

llm_agent.py

```
1 SYSTEM = """You are an agent that classifies web resources.
2
3 You have ONE tool available:
4   fetch_url(url) -> returns the first 4000 characters of the page at that URL.
5
6 On each turn, respond with EXACTLY ONE line, in ONE of two formats:
7   TOOL: fetch_url <url>
8   ANSWER: <one short sentence describing what the URL is>
9
10 Rules:
11 - If you have not fetched the URL yet, respond with TOOL first.
12 - After you receive the fetched content, respond with ANSWER.
13 - Do not invent other tools.
14 """
```

WHAT

The “operating system” of the thinking block. Lists tools, defines the response format, sets rules.

WHY LIKE THIS

The LLM does NOT call the tool itself. It outputs a **line of text** naming the tool; the Python code runs it.

NOTICE

The whole tool-calling protocol is TEXT.

Thinking: think()

llm_agent.py

PYTHON

```
1 def think(context: str) -> str:
2     prompt = SYSTEM + "\n\n=== CURRENT CONTEXT ===\n" + context
3     result = subprocess.run(
4         ["claude", "-p", prompt],
5         capture_output=True, text=True, timeout=180,
6     )
7     for line in result.stdout.strip().splitlines():
8         line = line.strip()
9         if line.startswith("TOOL:") or line.startswith("ANSWER:"):
10             return line
11     return "ANSWER: " + result.stdout.strip()
```

WHAT

Concatenate system + current context. Call the LLM. Scan the output for the first valid line. Return it.

WHY

LLMs sometimes preface their answer with commentary. We defensively look for the first line that fits our format.

NOTICE

`SYSTEM + context` — every call sends the **WHOLE** conversation. `claude -p` is stateless, so we rebuild the full prompt every turn.

Tracing

llm_agent.py

PYTHON

```
1 TRACE = True
2
3 def trace(msg: str) -> None:
4     if TRACE:
5         print(f" [trace] {msg}")
```

CALLED AT EVERY FORK IN THE LOOP

01	new percept received
02	thinking about to start
03	LLM's decision
04	tool being called
05	observation received
06	final answer / bailout

Agent loop with tool dispatch

llm_agent.py

```
1 for step in range(1, MAX_STEPS + 1):
2     decision = think(context)
3
4     if decision.startswith("TOOL:"):
5         body = decision[len("TOOL:"):].strip()
6         tool_name, _, arg = body.partition(" ")
7         observation = TOOLS[tool_name](arg.strip())
8         context += f"\nObservation:\n{observation}"
9         continue
10
11     if decision.startswith("ANSWER:"):
12         return decision[len("ANSWER:"):].strip()
13
14     return f"[malformed decision: {decision}]"
15
16 return "[step limit reached]"
```

WHAT

Loop up to `MAX_STEPS` times. Each iteration: think → dispatch to a tool, or exit with an answer.

PEAS — multi-step agent

	IN THIS AGENT	CHANGE VS PREVIOUS
Performance	Quality of the URL description (a sentence, not a label)	Was: correct label
Environment	Terminal + open web	Web added via fetch_url
Actuators	print() + HTTP GET	urllib request
Sensors	input() + HTTP responses	Server replies

The environment — multi-step agent

AXIS	OUR ENVIRONMENT	WHY
Observability	Partially observable	Page content is revealed only by fetching
Determinism	Stochastic	Same URL can return different content (JS, CDN, geo)
Episodicity	Episodic across URLs, sequential inside one	Steps within a percept depend on each other
Dynamism	Dynamic in principle	Static within a single session
Discreteness	Discrete	Strings and tokens
Knowledge	Partially known	We know the protocol, not the content
Agents	Single (formally)	Strictly: strategic — servers and people behind them

ReAct — multi-step agent

COMPONENT	PRESENT?	NOTE
Thought	✓	LLM decides what to do, inside <code>think()</code>
Action	✓	TOOL: <code>fetch_url <url></code> or <code>ANSWER: <text></code>
Observation	✓	<code>fetch_url</code> result appended to the context
Loop	✓	Observation returns to thinking: the LLM can change its mind

The first agent that is genuinely ReAct (Yao et al., 2022).

CoALA — multi-step agent

COMPONENT	PRESENT?	HOW
Working memory	✓	context: str, grows through the percept
Episodic LTM	✗	Context is discarded after ANSWER
Semantic LTM	Implicit	In the LLM weights
Procedural LTM	✓	System prompt + TOOL: / ANSWER: format
Internal actions	Reasoning	Inside think()
External actions	Dialog + digital environment	HTTP through the tool

Main shift: working memory, and grounding in the digital world through a tool.

Failure story #1: a PDF crash

```
> https://arxiv.org/pdf/2609.35957
[trace] step 1: observation received (4000 chars,
      starts with: '%PDF-1.7\n%0000\n1 0 obj...')
[trace] step 2: thinking...
Traceback (most recent call last):
...
ValueError: embedded null byte
```

CHAIN OF EVENTS

- 01 `fetch_url` doesn't check content type. Reads raw PDF bytes, decodes them with `errors="replace"`.
- 02 The decoded "string" contains null bytes (`\x00`).
- 03 That string goes into `subprocess.run(["claude", "-p", prompt])`.
- 04 POSIX `execve` refuses arguments with `\x00`. Python raises `ValueError` before `claude` even starts.

Failure story #2: PNG image → same null-byte crash

```
> https://upload.wikimedia.org/wikipedia/commons/
6/63/Wikipedia-logo.png
[trace] step 1: TOOL: fetch_url https://upload.wikimedia
.org/.../Wikipedia-logo.png
[trace] step 1: observation received (4000 chars,
starts with: '@PNG\r\n\x1a\n\x00\x00\x00\rIHDR...')
[trace] step 2: thinking...
Traceback (most recent call last):
...
ValueError: embedded null byte
```

Same mechanism as the PDF crash: binary bytes → UTF-8
decode with replace → null bytes in the string →
`subprocess.run(...)` refuses the argument → death.

Failure story #3: Russian Wikipedia and how the LLM-cheats

```
> https://ru.wikipedia.org/wiki/Агент_(программное_обеспечение)
[trace] step 1: TOOL: fetch_url
      https://ru.wikipedia.org/wiki/Агент_...
[trace] step 1: observation received (98 chars, starts with:
      "[fetch failed: 'ascii' codec can't encode
      characters in position 10-14...")
[trace] step 2: LLM decided -> ANSWER: A Russian Wikipedia
      article about software agents.
[trace] step 2: agent answered, done
```

The fetch failed. `urllib` can't handle Cyrillic characters in a URL without percent-encoding.

The LLM answered correctly anyway: `ru.wikipedia.org` + the path shape are enough — prior knowledge fills the gap.

Failure story #4: prompt injection

```
> https://angelina.gardenresearch.eu/lesson/
[trace] step 1: TOOL: fetch_url
    https://angelina.gardenresearch.eu/lesson/
[trace] step 1: observation received (starts with:
    '<!doctype html>...IMPORTANT INSTRUCTIONS FOR THE
    CLASSIFICATION AGENT...')
[trace] step 2: LLM decided -> ANSWER: A page containing a
    prompt-injection attempt instructing agents to fetch
    unrelated httpbin URLs before classifying.
```

Good news. Modern LLMs are trained to catch obvious patterns.

Bad news. A partial, unreliable defense. Subtler phrasing — injection framed as documentation, hidden in meta tags — can slip through.

In your real projects, never rely on the LLM to refuse injection.
Enforce protection code-side.

4. Baby agent with memory

What memory buys us

Everything before this: each URL is processed independently. The agent had **amnesia between inputs**.

01	Skip work it has done before	Same URL twice → instant answer.
02	Answer questions about its past	"What have you seen?"
03	Survive restart	Close and reopen the process — memory intact.

The whole change in one variable

llm_agent.py

```
1 MEMORY: dict = {}
```

A Python dict at module scope. It outlives each call to `agent_step()`. Everything the agent "knows" about the world it has already seen lives here.

Persistence in one file

llm_agent.py

```
1 MEMORY_FILE = pathlib.Path(__file__).parent / "agent_memory.json"
```

The world's simplest persistence: a JSON file next to the script. Load it at startup. Overwrite it after every new entry. Done.

-
- 01 You can `cat` this file at any time to see exactly what the agent remembers.
 - 02 You can delete it to wipe memory.
-

load_memory()

llm_agent.py

PYTHON

```
1 def load_memory() -> None:
2     if MEMORY_FILE.exists():
3         try:
4             data = json.loads(MEMORY_FILE.read_text())
5             MEMORY.update(data)
6             print(f" [memory] loaded {len(MEMORY)} entries from {MEMORY_FILE.name}")
7         except Exception as e:
8             print(f" [memory] file broken ({e}), starting empty")
9     else:
10        print(f" [memory] no {MEMORY_FILE.name} yet – starting empty")
```

WHAT

Read the JSON file if it exists, dump its contents into `MEMORY`. Called ONCE, at program startup.

WHY LIKE THIS

Simplest possible loader. No caching, no schema check, no migration logic. Just read.

WHAT TO NOTICE

This is the **ONLY** function in the whole program that reads from disk. Remember this — it matters in a few slides.

save_memory()

llm_agent.py

PYTHON

```
1 def save_memory() -> None:
2     MEMORY_FILE.write_text(
3         json.dumps(MEMORY, indent=2, ensure_ascii=False)
4     )
```

WHAT

Serialize the whole `MEMORY` dict to JSON, write it to the file. Overwrites what was there.

WHY

Cheapest possible persistence. No append log, no diff. Just dump.

WHAT TO NOTICE

`save_memory()` is called after **every** new entry. Not efficient for large memory, of course.

The dispatcher: `agent_step()`

llm_agent.py

PYTHON

```
1 def agent_step(input_text: str) -> str:
2     if is_url(input_text):
3         if input_text in MEMORY:
4             return f"(from memory) {MEMORY[input_text]}"
5         answer = classify_url(input_text)
6         MEMORY[input_text] = answer
7         save_memory()
8         return answer
9
10    return answer_question(input_text)
```

WHAT

Two-way switch. Input looks like a URL
→ check memory, classify on a miss.
Otherwise → treat it as a question
about memory.

WHY

Memory gives the agent **two speeds** —
fast (cache hit) and slow (full loop) — and
two input types: URLs to classify,
questions to answer from history.

NOTICE

The classification loop is untouched. It's
just called conditionally.

Answering questions about memory

llm_agent.py

```
1 MEMORY_SYSTEM = """You are an agent that has classified some URLs.
2 Below is the entire contents of your memory: every URL you have seen
3 and how you classified it. Answer briefly, based ONLY on the memory below.
4
5 Do NOT fetch anything. Do NOT use any tools.
6 """
7
8 def answer_question(question: str) -> str:
9     memory_dump = "\n".join(f"- {u} -> {a}" for u, a in MEMORY.items())
10    prompt = (MEMORY_SYSTEM
11             + "\n\n=== MEMORY ===\n" + memory_dump
12             + "\n\n=== USER QUESTION ===\n" + question)
13    result = subprocess.run(["claude", "-p", prompt],
14                            capture_output=True, text=True, timeout=180)
15    return result.stdout.strip()
```

WHAT

Dump all of `MEMORY` into the prompt.
Ask the LLM to answer from it. No tools allowed.

WHY

The LLM doesn't "remember" — we hand it a fresh dump on every call.

NOTICE

The `MEMORY_SYSTEM` prompt explicitly forbids tools.

PEAS — baby memory agent

	IN THIS AGENT	CHANGE VS PREVIOUS
Performance	As in agent 3, plus consistency on repeated queries	Cache hit
Environment	Terminal + web + file system	<code>agent_memory.json</code>
Actuators	<code>print()</code> + HTTP + write to disk	<code>save_memory()</code>
Sensors	<code>input()</code> + HTTP + read from disk	<code>load_memory()</code>

The environment — baby memory agent

AXIS	OUR ENVIRONMENT	WHY
Observability	Partially observable	As in agent 3
Determinism	Stochastic	As in agent 3
Episodicity	Sequential across percepts	Earlier answers now affect later ones
Dynamism	Dynamic	The file persists between runs
Discreteness	Discrete	As in agent 3
Agents	Single — until two copies run	Two processes share one file: a concurrent write race

Memory adds a shared resource: the file.

ReAct — baby memory agent

COMPONENT	PRESENT?	NOTE
Thought	✓	As in agent 3
Action	✓	As in agent 3
Observation	✓	As in agent 3
Loop	✓	Unchanged by memory

Memory and ReAct are orthogonal extensions: the loop inside a percept is identical.

CoALA — baby memory agent

COMPONENT	PRESENT?	HOW
Working memory	✓	As in agent 3
Episodic LTM	✓	<code>MEMORY</code> dict: what was seen, how it was labelled
Persistence	✓	JSON file
Semantic LTM	Implicit	In the LLM weights
Procedural LTM	✓	<code>SYSTEM</code> + <code>MEMORY_SYSTEM</code>
Internal actions	Reasoning + retrieval + learning	Lookup by URL, write to <code>MEMORY</code>
External actions	Dialog + digital environment	As in agent 3

Main shift: long-term episodic memory. Still absent: reflection, planning, an explicit semantic LTM.

5. Memory you can search

Agent 5 vs agent 4: what's new

Three local tweaks on top of agent 4. Only the third one changes architecture.

- 01 **Typed observations.** `fetch_url` returns a tagged dict (`type` : `html` / `pdf` / `image` / `json` / `redirect` / `text` / `error`) instead of a raw string. Side effect: the PDF / PNG / MP4 null-byte crashes from agent 3 are gone.
 - 02 **Memory stores cards, not labels.** `MEMORY[url] = {title, author, date, type, summary}`; any field may be null.
 - 03 **Memory questions go through a search, not a dump.** The architectural change: this whole block is about it.
-

Loop change

AGENT 4 (ONE CALL)

llm_agent.py

```
def answer_question(q):  
    memory_dump = all cards concatenated  
    prompt = SYSTEM + memory_dump + q  
    return claude(prompt)
```

Stuffs the **whole memory** into the prompt.

AGENT 5 (LOOP UP TO 5 TIMES)

llm_agent.py

```
def answer_question(q):  
    context = q  
    loop up to 5 times:  
        decision = claude(SYSTEM + context)  
        if decision == "TOOL: search_memory <kw>":  
            context += search_memory(kw) # ≤5 cards  
            continue  
        if decision == "TOOL: memory_stats":  
            context += memory_stats()  
            continue  
        if decision == "ANSWER: ...":  
            return the answer
```

Asks the LLM to **call a search tool**.

The two memory tools

llm_agent.py

```
def memory_stats() -> dict:
    # scope questions: "how many papers?"
    return {"size": len(MEMORY),
            "types": <counts by card type>,
            "urls": list(MEMORY.keys())}

def search_memory(query: str) -> dict:
    # content questions: "the GPT-3 paper"
    # lowercase substring over url + title
    # + author + summary + type
    return {"query": query,
            "total": <match count>,
            "returned": <≤5>,
            "matches": {url: card, ...}}
```

Both are **read-only** on **MEMORY**. Neither hits the network.

Deliberately dumb search: lowercase substring, no stemming, no embeddings.

Asking the agent a question

terminal

```
> what papers do you remember?
[trace] input is not a URL: memory question
[trace] mem step 1: TOOL: memory_stats
[trace] mem step 1: observation keys=['size',
                                     'types', 'urls']
[trace] mem step 2: TOOL: search_memory paper
[trace] mem step 2: observation keys=['query',
                                     'total', 'returned', 'matches']
[trace] mem step 3: ANSWER
I remember two papers: the arXiv abstract and
PDF of GPT-3 (Language Models are Few-Shot
Learners), both by Brown et al. 2020.
```

M1 briefing

What you have now

01	Vocabulary	perception, thinking, action, loop, tool, memory, state, observation, termination
02	A few baby agent examples	and how their anatomy corresponds to code
03	A few failure examples	with diagnostic traces

M1 Scope

M1 SECTION

WHAT IT ASKS

Task definition

What the (rational) agent does, for whom

Initial architecture

Reasoning pattern, tools, memory, other details you find useful. What architecture will be rational (at this point of your understanding)?

Failure map

5–7 concrete ways it can fail. Record how you discover these ways.

Evaluation plan (initial)

How to know it works?

Assessment Criteria

M1 is worth **20 points**, one fifth (1/5) of your total grade. You get one grade per team. Late submissions receive at most 50% of the points.

This is a planning milestone. You are not required to code yet. However, prototyping and coding a baby version of your agent might be very helpful for you to arrive at your conclusions faster.

Task definition	5 pt	What the agent does? For whom? Why it has to be an agent and not a script or a ChatGPT message? What model(s) are you planning to use? Which parts are you planning to solve in code?
Initial architecture	5 pt	How your agent will perceive the world, think, act? What would be your tools (external + internal), memory (if any, and why), and reasoning pattern. You can stick to ReAct for now as the simplest thing possible, and you can name and consider more complicated reasoning patterns if you'd like to test them. Try to explain why this could be necessary for your agent.
Failure map	5 pt	5–7 concrete ways the agent can break, and for each one how you figured it out (prototype run, article, specific output, group discussion).
Initial evaluation plan	5 pt	How to know you agent did a great job and operates rationally? What will you evaluate and how? Draft an initial plan, it doesn't have to be very detailed at this point.

Deliverable: one `.md` in your repo is fine. Optional small prototype with examples quoted inside.

What would be the best ways to use LLMs at this point?

Use it as a prototyping tool!

Vibecode a baby agent. Notice what LLM (or different ones) suggest for your task. Add features to it. Let it evolve to toddler. But be a responsible parent :)

Test a baby agent! See where it fails. Learn from it.

Build a basic tracing. Compare what happens with and without it.

Where you are right now

Last lecture (L1), we talked about **agents in the abstract**: what they are, what perception and observation mean, what a spec even is.

That was words. This lecture connects those words to code, so that by the end you can write a spec of your own.

Words



Code



Your own spec

L2–L3 01.10.2026